

BINARY SEARCH TREES

Problem Solving with Computers-II

C++

```
#include <iostream>
using namespace std;

int main(){
    cout<<"Hola Facebook\n";
    return 0;
}
```

Imagine you're designing a system for a stock exchange.



Real-Time System: Traders submit buy/sell orders in real-time — thousands per second.

Order Book: List of buy orders, sorted by price (highest to lowest), matches orders at the same price.

Key Operations: Insert new orders, search for matches, print in sorted order, find the best price.

BUY	SELL
"I'll buy 100 shares at \$150"	"I'll sell 50 shares at \$150"
ORDER BOOK	
SHARES	PRICE
200	155
150	152
100	150

To manage the buy/sell order book, which data structure should we use?

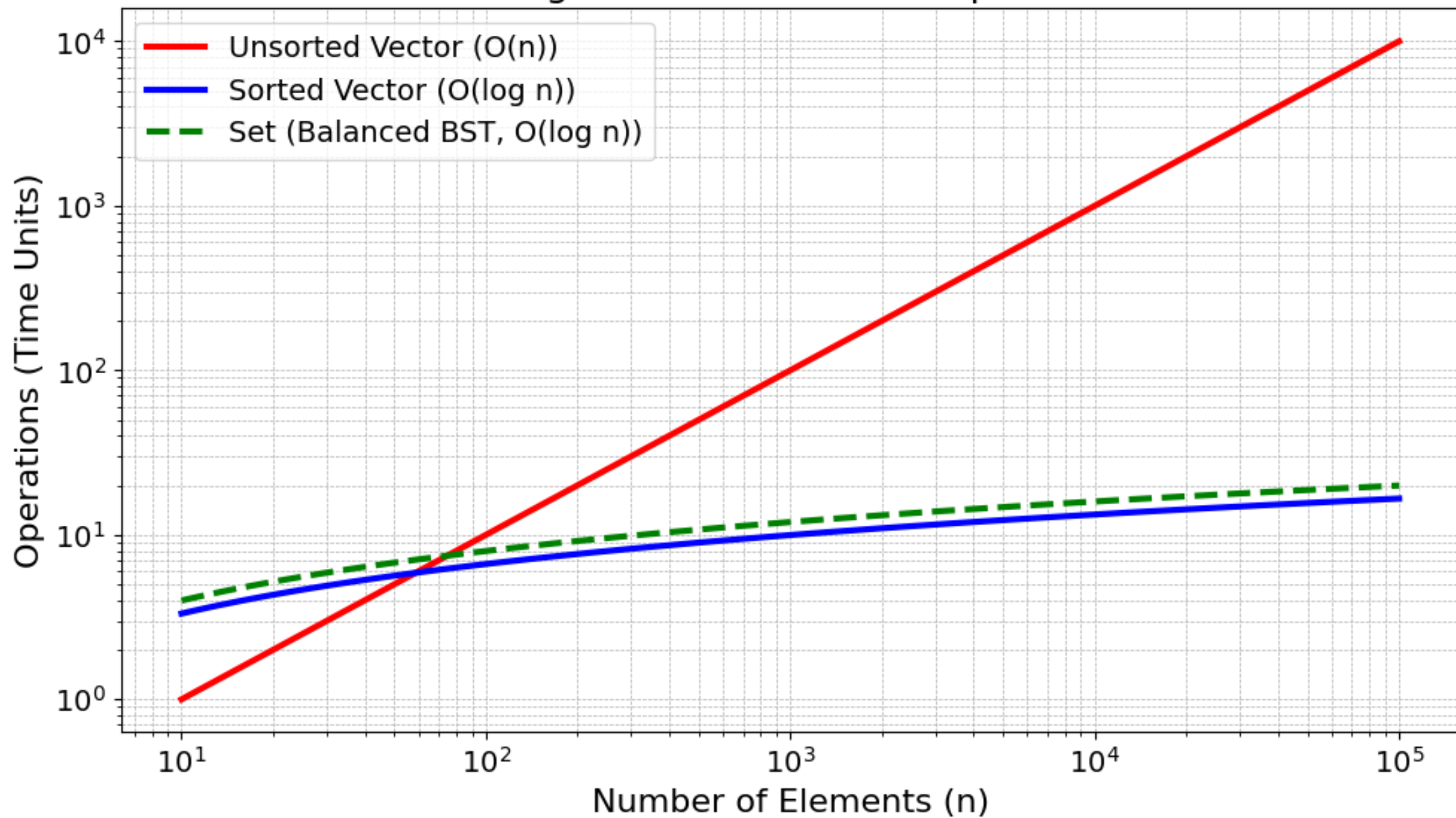
To manage the buy order book, which data structure should we use?

The system needs to handle a stream of new orders, search for prices to match trades, print orders in sorted order, and find the best price—all as fast as possible.

Discuss with your peers and vote for the best option:

- A) **Unsorted Vector**: Add orders with `push_back`, search with `std::find`, sort when printing.
- B) **Sorted Vector**: Add orders, re-sort after each addition, search with `std::binary_search`.
- C) **`std::set`**: Add orders with `insert`, search with `find`, print in order, get best price with `begin()`.
- D) **Array**: Fixed-size array, manually manage sorting and searching.
- E) More than one options works!

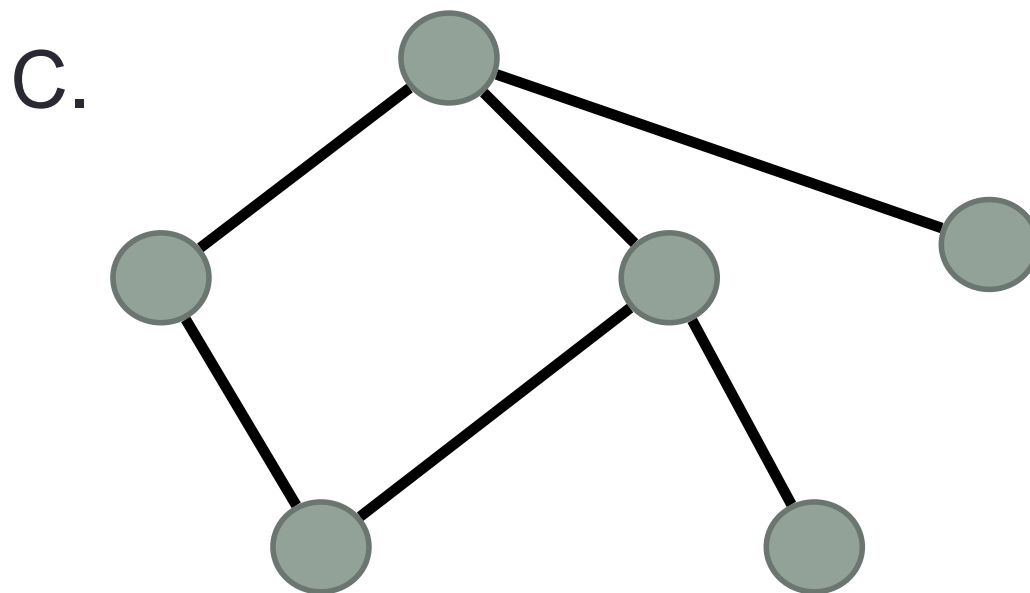
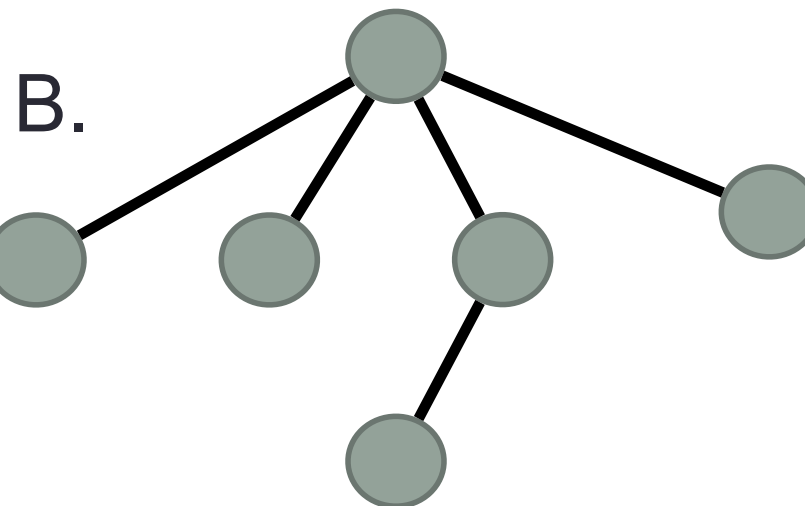
Scaling of Worst-Case Find Operations



Binary Search Trees (std::set)

- What are the operations supported?
- What are the running times of these operations?
- How do you implement the BST i.e. operations supported by it?

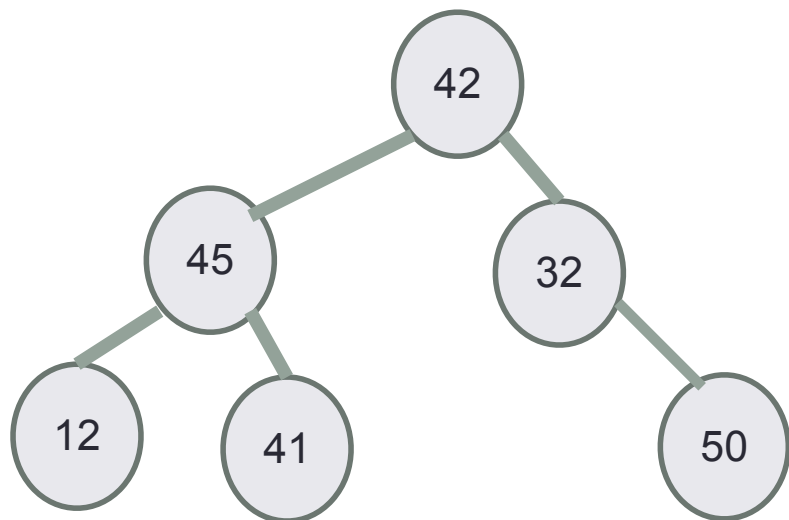
Which of the following is/are a tree?



D. A & B

E. All of A-C

Binary Trees



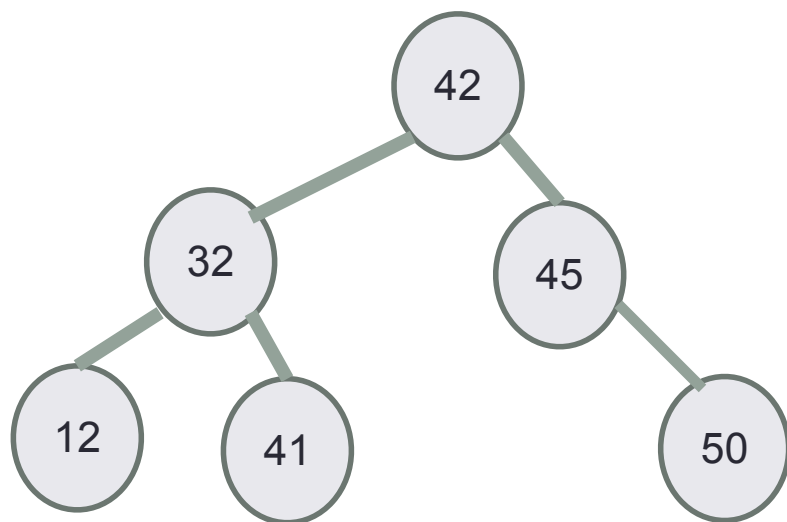
In a tree, nodes are arranged in a hierarchy

- One node is distinguished as the root
- Each node:
 - stores a key (k)
 - has a pointer to child nodes and parent (optional)
- Unique path between any two nodes
- Leaf nodes have no children

In a binary tree, each node has at most _____ children

```
struct TreeNode {  
  
    TreeNode* left;  
    TreeNode* right;  
    TreeNode* parent;  
    int const data;  
  
    TreeNode(int d) : data(d) {  
        left = right = parent = nullptr;  
    }  
};
```


Binary Search Tree – What is it?

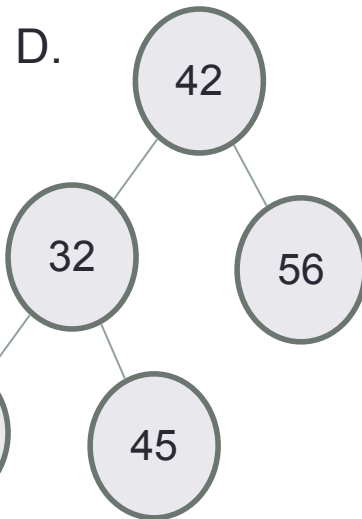
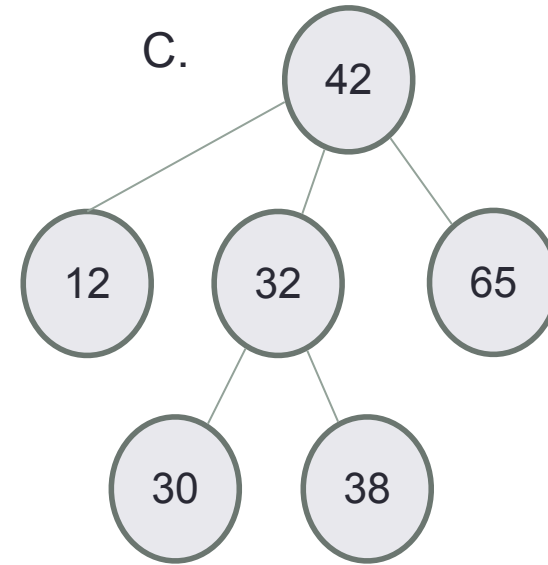
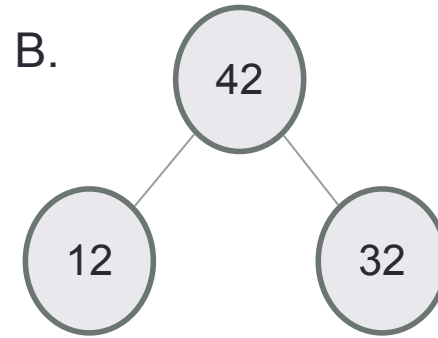
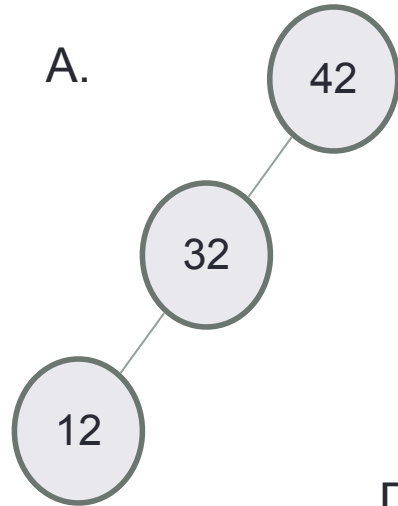


BST is a binary tree where each node satisfies the **Search Tree Property**

For any node,
Keys in node's left subtree < Node's key <
Keys in node's right subtree

Do the keys have to be integers?

Which of the following is/are a binary search tree?



E. More than one of these

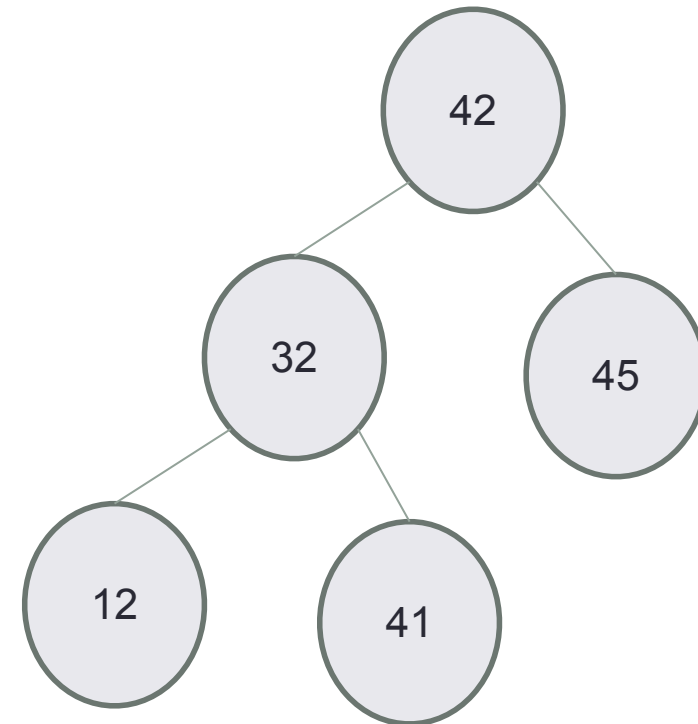


- Path – a sequence of (zero or more) connected nodes.
- Length of a path - number of edges traversed on the path
- Height of node – Length of the longest path from the node to a leaf node.
- **Height of the tree** - Length of the longest path from the **root** to a leaf node.

BSTs of different heights are possible with the same set of keys
Examples for keys: 12, 32, 41, 42, 45

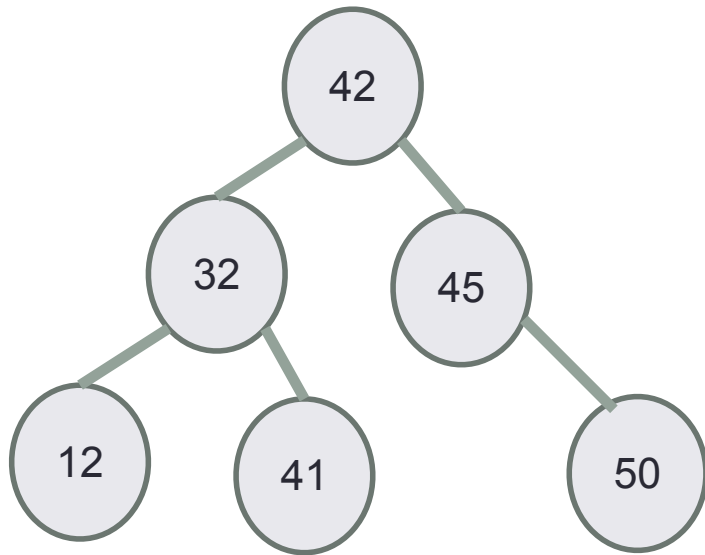
Define the BST ADT

Operations
Search
Insert
Min
Max
Successor (next largest key)
Predecessor (next smaller key)
Delete
Print elements (3 variations)



Write a function to create a small BST manually (not using insert!)

BSTs allow efficient search!

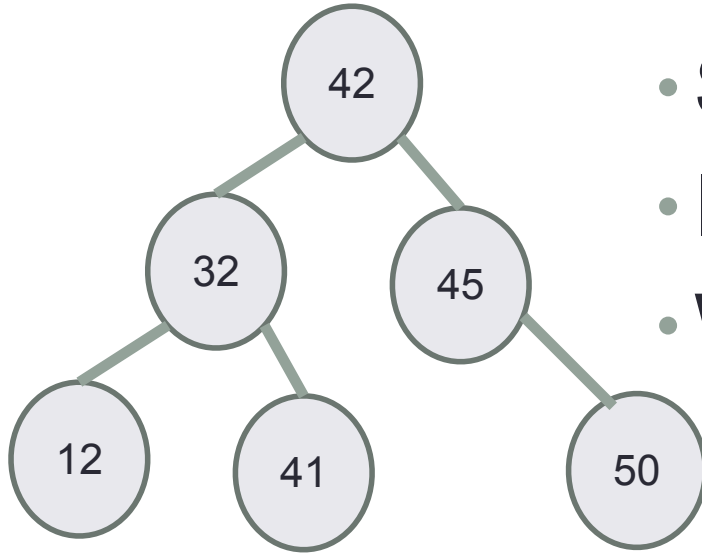


- Start at the root;
- Trace down a path by comparing **k** with the key of the current node **x**:
 - If the keys are equal: we have found the key
 - If $k < \text{key}[x]$ search in the left subtree of **x**
 - If $k > \text{key}[x]$ search in the right subtree of **x**
- What is the running time of search?



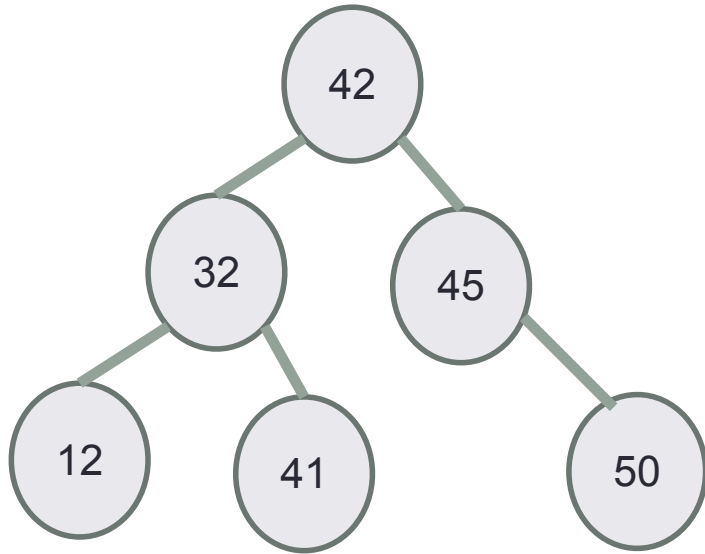
Search for 41, then search for 53

Insert



- Insert 40
- Search for the key
- Insert at the spot you expected to find it
- What is the running time of insert?

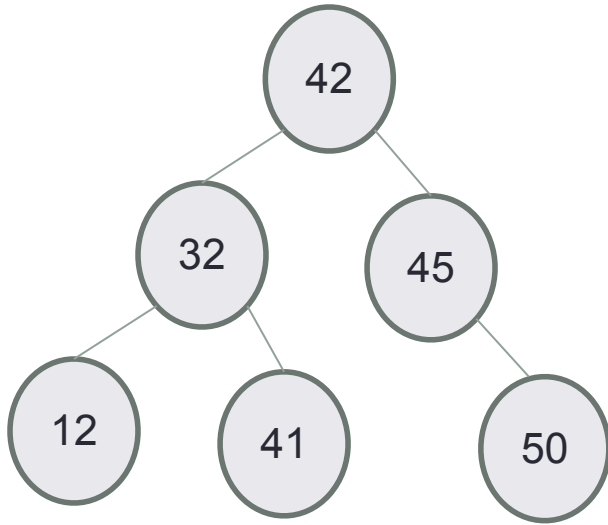
Min/Max



Which of the following describes the algorithm to find the maximum value in the BST?

- A. Return the root node's value
- B. Follow **right child** pointers from the root, until a node with no right child is encountered, return that node's key
- C. Follow **left child** pointers from the root, until a node with no left child is encountered, return that node's key

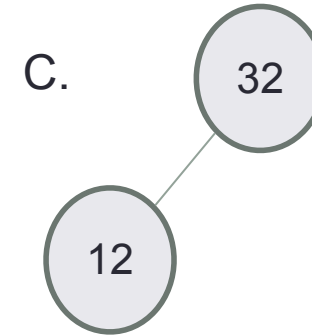
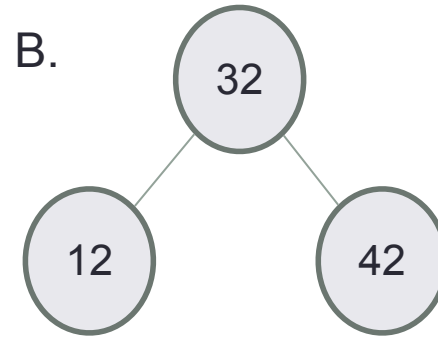
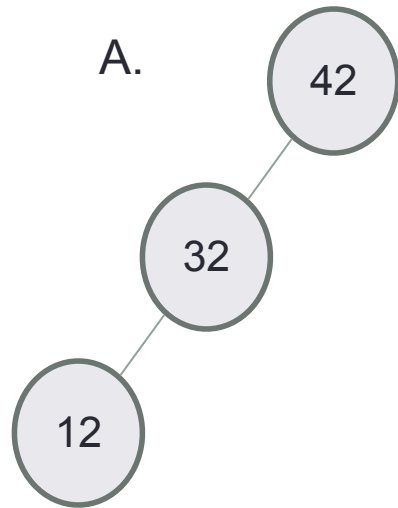
In order traversal: print elements in sorted order



Algorithm Inorder(tree)

1. Traverse the left subtree, i.e., call Inorder(left-subtree)
2. Visit the root.
3. Traverse the right subtree, i.e., call Inorder(right-subtree)

Write a member function for the BST ADT to compute its height



```
int bst::getHeight() const{  
    return getHeight(root);  
} // returns the height of the tree
```